CYBERSEAT

Alibi | Initial-evaluation deliverables

October 23, 2026 - Software Engineering, SE-2404

Proposal / Literature / Technology / Requirements / Architecture

Prepared for team review. Design-stage material; no implemented product, executed experiment or new human approval is claimed.

ECR-2: Project proposal and scope

ECR-4: Critical review - 13 sources, 9 scientific publications

ECR-6: Technology alternatives and proposed decisions

ECR-7: 16 booking and 10 non-functional requirements

ECR-10: Context and component diagrams, transaction protocol

Editable documents and diagram sources accompany this PDF in the GitHub branch. This package covers Alibi's five assigned analysis/design tasks, not the full team submission.

CyberSeat - Project Proposal and Scope

Version 1.0 | October 23, 2026 | Owner: Aibekuly Alibi | ECR-2

Status: prepared for team review. This is a design-stage proposal, not evidence of an implemented or evaluated product.

1. Project identity

Title: Information System for Managing Gaming Station Reservations in Esports Centres.

Programme and group: Software Engineering, Astana IT University, SE-2404. **Supervisor:** Myrzakerimova Alua. The names and responsibilities below follow the current repository README.

Participant	Diploma responsibility	Initial-evaluation contribution
Aibekuly Alibi	Shared database and reservation module	Proposal, literature review, technology comparison, booking and non-functional requirements, system context and component diagrams
Mansur Urynbassar	Visitor and administrator interfaces	User evidence, existing-system comparison, scenarios, UX mockups and use-case diagram
Akhmetov Nurali	Administrative server functionality	Administrative requirements, access matrix, evaluation plan, risks and supervisor-feedback record

2. Problem and evidence boundary

The working problem hypothesis is that separate telephone, messaging and spreadsheet records make it difficult to give visitors consistent station availability and administrators a consolidated schedule. Two users may both see a station as available before either reservation is saved. Independent administrative changes can also invalidate a reservation if schedule and maintenance rules are checked inconsistently.

These are plausible failure modes, not measured facts about a particular club. No interview count, client endorsement, market size or reduction in manual work is asserted here. ECR-3 must supply the actual user/client evidence and may require changes to this proposal. Literature supports the concurrency problem and design methods; it does not establish local demand.

Target users: guests comparing clubs; registered visitors reserving PC or console stations; administrators operating assigned clubs; a system administrator maintaining clubs and administrator assignments. A multi-club manager can be represented through assignments to several clubs, without an unrestricted extra role.

3. Aim, research questions and objectives

Aim: design, implement and evaluate a web information system that combines gaming-station discovery, interval reservations and club administration while preserving reservation consistency and role-based access boundaries.

RQ1: Can the proposed transaction protocol preserve non-overlapping active reservations and maintenance intervals under concurrent operations and retries? **RQ2:** Can target users complete discovery, reservation and cancellation tasks through the proposed interface? **RQ3:** Does a shared schedule provide administrators with correct, reproducible utilisation information?

Objective	Deliverable	Planned evidence of achievement
O1 Identify users, needs and scope	Evidence record and bounded proposal	Trace each claimed need to a real interview/client record or label it an assumption
O2 Examine research and alternatives	Critical review, product comparison, technology decisions	Cited sources, limitations and explicit requirement implications
O3 Specify verifiable behaviour	Requirements, access matrix and scenarios	IDs, priorities, acceptance criteria and cross-document traceability
O4 Design interactions and architecture	UX mockups, context, components and use cases	Consistency check against scenarios, data ownership and failure flows
O5 Build the core workflow after initial evaluation	Web client, API, relational schema and migrations	Demonstration of discovery, creation, viewing and cancellation
O6 Evaluate correctness and usability	Reproducible tests and participant task observations	Invariant checks, access tests, load conditions and task results; no invented outcomes

4. Included and excluded scope

Included	Boundary
Club and station catalogue	PC and console specifications, club address, working hours, filters and station layout
Availability search	One station and a finite visit interval; club-local display; server decides at commit
Personal reservations	Authenticated creation, own-record viewing, cancellation before start and retained history
Club administration	Assigned-club stations, schedules, maintenance intervals, reservations, events and reports
Role-based access	Guest, visitor, club administrator and system administrator; enforced by the server
Event calendar	Informational published events; browsing does not create a reservation or tournament entry
Reporting	Defined utilisation measures from controlled booking and schedule data; no financial settlement

Excluded: online payments, loyalty programmes, hardware/device control, native mobile applications, automatic tournament registration and paid ticketing. External tournament feeds, email/SMS and social login are not required dependencies. Administrators can enter event information manually with its source and publication status. Group booking across multiple stations is deferred; the core transaction reserves one station.

5. Proposed solution and critical constraints

A React/TypeScript web client communicates with a FastAPI REST API. The API is a modular monolith with access, catalogue, reservation, administration, events and reporting boundaries backed by PostgreSQL. These are proposed choices, justified in the technology comparison, not installed or benchmarked services.

The central invariant is that confirmed reservations for the same station never overlap. A reservation must also respect working hours and maintenance. Checking the browser display alone cannot preserve this invariant. Creation, cancellation, maintenance and schedule changes must share a database transaction protocol. Repeated submission of one logical creation operation must not create another booking. An unknown network outcome must remain unknown until reconciled.

The design uses half-open intervals [start, end), explicit club time zones and UTC instants in storage. A booking ending at 15:00 may be followed by one starting at 15:00. A detailed schema is deferred to ECR-12; these semantics are already required for the October design.

6. Delivery stages and evaluation

Stage	Exit condition
Initial submission, October 24, 2026 before 18:00 according to the university's stated time	Analysis, cited research, requirements, diagrams, UX and evaluation plan assembled; outstanding evidence/review clearly identified
Foundation	Schema, migrations, authentication and catalogue can be demonstrated on synthetic data
Core MVP	Reservation creation, own history, cancellation, maintenance coordination and assigned-club access pass integration tests
Diploma extension	Event calendar, station layout, defined reports and audit trail are integrated
Final evaluation	Reproducible correctness and load results, usability observations, limitations and demonstration are reported

No final experiment results are required or claimed at the initial submission. Planned correctness checks include simultaneous conflicting requests, adjacent intervals, retries, cancellation races, maintenance races and cross-club access attempts. Performance targets and workload conditions are specified in ECR-7 requirements; Nurali's ECR-13 plan should adopt or explicitly revise them. Usability assessment should combine task success, time, observed errors and a validated questionnaire, rather than treating a questionnaire score as proof of all quality attributes.

7. Assumptions, dependencies and review

The project depends on access to target users for ECR-3, team agreement on administrative rules in ECR-8, a hosting environment for the eventual prototype and supervisor review recorded in ECR-16. The main technical risks are interval races, inconsistent authorisation and scope growth; mitigations are shared transactional writes, negative access tests and the exclusions above. Real personal data is unnecessary for the October package.

This proposal aligns with the current README and approved ECR-1 scenarios. Reviewer requested: Mansur for user flow/event scope and Nurali for administrative scope and evaluation alignment. Their review of this new proposal has not yet been recorded. Supervisor approval is not implied.

Related deliverables

- Critical literature review and source matrix
- Technology alternatives and decisions
- Booking and non-functional specification
- Architecture, diagrams and transaction protocol
- User and administrator scenarios

CyberSeat - Critical Literature Review

Version 1.0 | October 23, 2026 | Owner: Aibekuly Alibi | ECR-4

Status: prepared for team review. This focused engineering review supports the booking module and its quality requirements. It is not a systematic review, a market study or evidence of user interviews.

1. Questions and selection method

The review asks: how can overlapping reservations be prevented under concurrency; how should correctness be tested; and how can access control and usability be evaluated without overstating the evidence?

Searches on October 23, 2026 used publisher/conference pages, author-hosted papers and official technical specifications. Query families included database isolation verification, concurrent application integrity, snapshot isolation, interval constraints, object-level authorisation and SUS analysis. Papers were included when they provided a method, empirical finding or explicit limitation relevant to a requirement. Vendor marketing, unsourced summaries and publications whose identity could not be verified were excluded from the source matrix. No exhaustive search count or screening flow is claimed.

The matrix contains **13 substantively used sources: 9 scientific publications and 4 technical/standards sources**. Six of nine scientific publications are from 2022-2025, within the recent five-year window. Three older papers are retained for foundational isolation and application-integrity arguments. Current specifications are used for concrete implementation behaviour; papers about older releases are not treated as descriptions of current software versions. Where a source was available only as a publisher abstract, the matrix says so and avoids unsupported detailed performance claims.

2. Critical source matrix

ID and citation	Approach and reported result	Limitation / transfer boundary	CyberSeat implication
S01 Berenson et al. (1995), A Critique of ANSI SQL Isolation Levels, SIGMOD, pp. 1-10. Author-deposited paper, DOI 10.1145/223784.223785	Analyses anomaly-based isolation definitions and defines snapshot isolation; shows why isolation labels alone are insufficient to characterise behaviour	Foundational semantics, not a booking-system evaluation; 2007 arXiv deposit is not the original publication year	Specify forbidden committed states explicitly rather than saying the database is merely ACID; BR-04, NFR-01
S02 Ports and Grittner (2012), Serializable Snapshot Isolation in PostgreSQL, PVLDB 5(12), pp. 1850-1861. Paper	Describes PostgreSQL SSI implementation and benchmark evaluation; conflicting executions may abort to preserve serialisability	Concerns the introduction of SSI in PostgreSQL 9.1; benchmark outcomes do not predict CyberSeat latency	Full-transaction retry is necessary if SSI is selected; explicit guard locking is another candidate, not equivalent to assuming no aborts; NFR-05
S03 Bailis et al. (2015), Feral Concurrency Control: An Empirical Investigation of Modern Application Integrity, SIGMOD, pp. 1327-1342. Publisher abstract	Empirically studies ORM-backed integrity mechanisms and identifies concurrency-related integrity failures in some application checks	Ruby on Rails sample; the publisher abstract supports the general finding, not a claim about this Python project	A check followed by an independent insert is insufficient protection; add database enforcement and simultaneous-operation tests; BR-04/09
S04 Huang et al. (2023), Efficient Black-box Checking of Snapshot Isolation in Databases, PVLDB 16(6), pp. 1264-1276. Paper, DOI 10.14778/3583140.3583145	PolySI characterises snapshot isolation using generalised polygraphs, checks histories with SMT and reports counterexamples; evaluation reproduces known anomalies and finds new violations	Checks an isolation property under its history model; does not establish business invariants or model the complete reservation API	Preserve operation histories and make counterexamples understandable; checking HTTP status counts alone cannot establish consistency; NFR-01/03
S05 Liu et al. (2024), Leopard: A General Test Suite for Isolation Level Verification, CIDR. Paper	Collects client-side interval traces and mirrors concurrency-control mechanisms for black-box checking; reports bugs in commercial DBMSs	Verification depends on the model and collected observations; building a general DBMS checker exceeds diploma scope	Record start/end times, results and final DB state for controlled races; use a small domain oracle rather than claiming formal DBMS verification; NFR-01
S06 Li et al. (2024), DBStorm: Generating Various Effective Workloads for Testing Isolation Levels, ISSTA. Conference abstract, DOI 10.1145/3650212.3680318	Generates diverse conflict patterns and anomaly-sensitive workloads; reports finding DBMS implementation bugs	Conference abstract was used; no full-method replication or numerical speedup is claimed. DB engine anomalies differ from application rule violations	Vary conflict shapes: identical, partial, containing, adjacent and cross-operation intervals; repeating one happy path is inadequate; BR-04/05/09
S07 Liu, Gu, Wei and Basin (2024), Plume: Efficient and Complete Black-Box Checking of Weak Isolation Levels, PACMPL 8(OOPSLA2), Article 302. Author paper, DOI 10.1145/3689742	Uses fine-grained anomaly patterns and clock techniques to check representative weak-isolation guarantees, with informative counterexamples	Read committed, read atomicity and causal-consistency checks are not proofs of serialisability or of non-overlapping bookings	Name the property being checked. Domain invariant tests must remain even if the database satisfies its configured isolation contract; NFR-01

ID and citation	Approach and reported result	Limitation / transfer boundary	CyberSeat implication
S08 Møldrup and Pavlogiannis (2025), AWDIT: An Optimal Weak Database Isolation Tester, PACMPL 9(PLDI), Article 209. Author paper, DOI 10.1145/3742465	Develops algorithms and complexity bounds for history checking of RC, RA and CC, with comparative evaluation	Optimality is conditional on the paper's problem model and complexity assumptions; weak-isolation validity is weaker than application correctness	Keep repeatable, finite histories and explicit oracles; do not infer no-overlap merely from selecting a named isolation level; NFR-01
S09 Blattgerste, Behrends and Pfeiffer (2022), A Web-Based Analysis Toolkit for the System Usability Scale, PETRA, pp. 237-246. Author paper, DOI 10.1145/3529190.3529216	Presents an open-source toolkit for calculating, plotting and contextualising SUS scores	A toolkit is not an esports-club study. SUS reflects perceived usability; its 0-100 scale is not a percentage of successful tasks	Combine SUS with observed task completion, time and errors; disclose participant selection and small sample limitations; NFR-07
S10 PostgreSQL Global Development Group, Range Types, PostgreSQL 18 documentation. Specification	Documents range bounds, overlap operators and exclusion constraints, with scheduling examples	Enforces configured constraints, not all cross-table rules; does not establish that the application uses them correctly	Use half-open timestamp ranges and a same-station overlap constraint; separately coordinate maintenance and working-hours changes; BR-04/05/09/10
S11 PostgreSQL Global Development Group, Explicit Locking, PostgreSQL 18 documentation. Documentation	Describes row locks, transaction-scoped lock lifetimes and deadlock considerations	A lock protocol works only if every relevant writer follows it; more contention can increase latency	Define club/station guard rows, universal lock order and bounded retries; a process-local lock is insufficient across API workers; BR-09/10, NFR-05
S12 OWASP (2023), API1: Broken Object Level Authorization. Guidance	Explains resource-ID manipulation and the need for object-level authorisation on requested objects	Risk guidance, not an exhaustive security certification or measured probability for this system	Check ownership and club assignment in every server operation; test foreign IDs and cross-club requests; BR-07/08, NFR-02
S13 W3C (2024), Web Content Accessibility Guidelines 2.2, Recommendation of December 12, 2024. Standard	Provides testable accessibility criteria, including keyboard interaction, text alternatives and visible focus	Meeting selected criteria is not proof of full AA conformance or general usability; complete assessment remains future work	Use text in addition to colour, labelled errors and keyboard checks for core screens; BR-13, NFR-06

All links were consulted on October 23, 2026. The matrix is a critical engineering synthesis; its CyberSeat implications are proposed design conclusions, not claims made by the original authors about CyberSeat.

3. Synthesis: integrity is a business property

S01 explains the semantic gap between isolation labels and the actual anomalies an application must prevent. S03 provides an empirical reason to distrust uncoordinated application checks. For a station reservation, two individually valid reads of an empty schedule can precede conflicting inserts. The resulting design conclusion is to state an invariant over the committed records, enforce it in the database and test it through independent connections.

S02 and S11 offer different coordination mechanisms: serialisable transactions with retry, or explicit transaction-scoped locks. CyberSeat provisionally chooses a shared lock protocol plus a database overlap constraint. This reduces the amount of implicit predicate-level reasoning for the first implementation, at the cost of serialising some operations. It is an engineering choice to benchmark later, not a demonstrated superiority claim.

S10 directly supports interval representation. However, an exclusion constraint on reservations cannot, by itself, prevent a reservation from overlapping a maintenance record stored in another table. The architecture therefore requires reservation and maintenance writers to lock the same station guard before validation and write. Working-hours changes and reservations similarly coordinate through a club guard. ECR-8 must preserve this protocol when administrative endpoints are designed.

4. Synthesis: tests need histories and diverse conflicts

Recent work S04-S08 demonstrates several ways of checking transactional histories or generating revealing workloads. Their guarantees differ, so citing a successful weak-isolation checker would not prove that CyberSeat never double-books a station. Nor is it proportionate to implement a new general-purpose isolation checker for this diploma.

The adopted scope is domain-specific: release requests through a start barrier, preserve their input/response/operation identity, then query the committed state using an overlap oracle. Add a forced network interruption after commit, restart/reconciliation checks and booking-versus-maintenance races. A small workload that produces a comprehensible failing history is more useful here than an unsupported claim of production-scale reliability.

5. Synthesis: user quality and access are separate questions

S09 supports systematic calculation and interpretation of a perceived-usability measure. S13 gives concrete accessibility checks. Neither establishes demand for an esports booking service; that remains ECR-3. Prototype task observations should therefore be reported alongside questionnaire results, with participant recruitment and limitations disclosed.

S12 requires a server authorisation decision for each resource operation. UI hiding is insufficient: tests should change reservation IDs and club IDs directly. Ownership and assigned-club boundaries must remain effective even when a visitor understands the API. Reports must distinguish scheduled utilisation from actual attendance, which the proposed system cannot observe without additional evidence or hardware.

6. Research-to-requirement decisions and remaining gaps

Finding	Design decision	Requirement / planned verification
Integrity cannot be inferred from labels or ORM validation	Database overlap constraint and common transactional write path	BR-04/09/10, NFR-01; real-DB race tests
Cancel/retry/network failure alters the observable history	Durable operation identity and result reconciliation	BR-06/11, NFR-03; response-loss and retry tests
Scheduling uses interval rather than scalar uniqueness	[start,end) and club-zone conversion	BR-05/14; adjacency, containment and overnight fixtures
Testing only one conflict pattern misses behaviours	Controlled family of cross-operation histories	NFR-01; final-state oracle after each run
Resource access depends on actor and object	Server owner/assignment checks	BR-07/08, NFR-02; foreign-resource negative tests
Perceived usability is only one quality dimension	Task metrics plus questionnaire and accessibility checks	NFR-06/07; formative prototype study

Important gaps remain: no verified local demand or club workflow observations; no executed CyberSeat benchmark; no chosen final authentication mechanism; no complete accessibility audit; and limited domain-specific empirical research in this focused review. ECR-3 and ECR-5 must add field evidence and product comparison. Mansur is requested to review the UX interpretation; Nurali is requested to review authorisation and reporting implications. Their review of this document is pending.

CyberSeat - Technology Comparison and Decision Record

Version 1.0 | October 23, 2026 | Owner: Aibekuly Alibi | ECR-6

Status: proposed decisions, prepared for team review. No technology benchmark, implementation or newly obtained teammate approval is claimed. The baseline matches the current README.

1. Evaluation basis

Priorities are reservation integrity (BR-04/09), explicit authorisation (NFR-02), reproducible testing (NFR-01/08), a browser-based interface and manageable integration for a three-person diploma team. Ratings below are qualitative engineering judgements, not measurements. Team expertise has not been independently measured and is not used to invent numerical scores. All candidates can support a correct design; none makes the reservation transaction correct automatically.

2. Server platform: FastAPI, Django and NestJS

Criterion	FastAPI / Python	Django / Python	NestJS / TypeScript
API contract and validation	Type-based request/response models and OpenAPI integration	Structured web framework; an API layer can be added, with serializer and contract choices to make	Controllers, DTO validation, modules and OpenAPI integration available
Administrative foundation	Custom admin screens, authorisation and session integration required	Built-in admin, authentication and ORM reduce common administrative setup	Guards and modules support access policies; persistence and admin UI still chosen separately
Booking consistency	SQLAlchemy/driver transaction and PostgreSQL constraints required	ORM transactions available; domain rules and appropriate constraints still required	ORM/driver transactions required; JavaScript execution does not solve cross-worker races
Team integration	Clear typed HTTP contract with React; two languages across client/server	Strong fit if conventional data management dominates	Shared TypeScript language across client/server; language familiarity remains to be confirmed
Main tradeoff	Lean API construction, but more security/admin composition work	More integrated conventions; custom booking workflow still needs application-specific design	More module/DTO setup and a different backend toolchain from the current baseline

Proposed decision D1: retain FastAPI because a separately developed React client and explicit reservation API are central to the current design. Use request validation and generated API documentation to reduce integration ambiguity. Do not infer a latency advantage from vendor feature descriptions. Django is a credible fallback if the team prefers its integrated administrative foundation. NestJS becomes preferable if confirmed backend TypeScript expertise outweighs switching cost. Revisit D1 before implementation if Nurali cannot support the proposed security/admin composition.

Primary sources: FastAPI features, Django transactions, Django admin, NestJS documentation. Consulted October 23, 2026; versioned pages are reference editions, not claims of the newest available release.

3. Storage: PostgreSQL, MySQL and MongoDB

Criterion	PostgreSQL	MySQL / InnoDB	MongoDB
Time-interval model	Native timestamp ranges and overlap operators	Start/end columns with explicit overlap predicates	Start/end fields and application-defined predicates
Declarative no-overlap rule	GiST exclusion constraint can combine station equality and range overlap	Scalar unique keys do not express arbitrary interval overlap; use an explicit transactional locking design	Unique indexes likewise do not express arbitrary interval overlap; schema and transaction protocol must prevent conflicts
Transactions and coordination	Row locks and serialisable isolation; range constraint complements the application protocol	InnoDB transactions and locking support a correct solution when consistently applied	Single-document atomicity and multi-document transactions supported; deployment and data modelling choices matter
Relationships/reporting	Foreign keys and SQL fit clubs, stations, owners, assignments and utilisation queries	Relational model also fits the domain well	Flexible documents, but related booking/schedule/reporting records require explicit consistency design
Main tradeoff	PostgreSQL-specific range features and operational setup	More custom interval-integrity logic to demonstrate	Less direct fit for this relational scheduling invariant; not excluded because of a false lack-of-transactions claim

Proposed decision D2: PostgreSQL. Its interval constraint makes the central invariant visible and independently enforceable. tstzrange values use) bounds; btree_gist may be needed for station equality in the exclusion index. A partial constraint applies only to CONFIRMED reservations. This covers booking-versus-booking overlap, not maintenance in another table or working-hours containment. Those rules require the shared guards described in the [architecture. MySQL and MongoDB remain viable with an alternative proven protocol; no unsupported performance ranking is asserted.

Primary sources: PostgreSQL 18 range types, PostgreSQL 18 locking, MySQL 8.4 InnoDB locking, MongoDB transactions. Consulted October 23, 2026.

4. Component interaction: REST, GraphQL and gRPC

Criterion	REST over HTTP with JSON	GraphQL	gRPC
Browser integration	Direct fetch/client calls with explicit endpoints	Browser clients can request shaped data through a typed schema	Browser use needs a compatible web integration; native gRPC targets service RPC
Contract	OpenAPI documents operations, bodies and status codes	Schema plus query/mutation validation	Protocol Buffers and generated service/client contracts
Core workflow fit	Catalogue reads and create/cancel commands are straightforward	Flexible catalogue composition, but per-field authorisation and query limits need attention	Strong typed service interaction; little current need for internal service RPC in a monolith
Failure/retry semantics	Explicit HTTP statuses and application operation key	Domain errors and retry identity still application responsibilities	Deadlines and status model help RPC; retries still need business-level deduplication
Complexity for this scope	One browser API and generated documentation	Additional resolver/query design without an established need	Additional browser transport/tooling without planned independent backend services

Proposed decision D3: REST with a versioned /api/v1 contract and OpenAPI. Use 409 for a valid request conflicting with current state, 422 for validation and 401 for missing/expired authentication. Foreign own-reservation IDs return 404. POST creation is made replay-safe by the application protocol, not by the HTTP method itself. Availability refresh is sufficient for the core scope; WebSockets or server push can be reconsidered if user evidence establishes a real need.

Sources: HTTP semantics, RFC 9110, GraphQL specification, October 2021 reference edition, gRPC introduction. Consulted October 23, 2026. These sources describe mechanisms; the suitability judgement is this project's decision.

5. Client and deployment alignment

D4, existing baseline: React, TypeScript and Vite for visitor/admin interfaces. React's component model and TypeScript's static checks fit reuse across catalogue, forms and admin views, but do not validate incoming data or authorise requests. Vue with TypeScript is a credible component-based alternative; server-rendered templates reduce a separate client build but fit less directly with the existing interface ownership and API plan. Keeping the documented baseline avoids a speculative redesign. Sources: React documentation, TypeScript handbook, Vue guide.

React/TypeScript appears in the current README and ECR-1 interface baseline. **Mansur's explicit confirmation of this comparison and its tradeoffs is still pending**; no new agreement is claimed. Before implementation, request review of forms, error handling, API models and accessibility behaviour.

D5, provisional tooling: SQLAlchemy with Alembic migrations, Pytest for real-DB integration tests, Playwright for browser scenarios and k6 for load generation, consistent with the README. Pin compatible versions only after the first reproducible environment is validated. Docker Compose will describe the local client/API/database environment. It is sufficient for a demo, but does not establish production availability, backups or disaster recovery.

6. Cross-cutting decisions and reconsideration triggers

Decision	Requirement served	Cost / risk	Revisit when
Modular monolith with one transactional database	BR-04/09, NFR-08	Coupled deployment and shared schema	Independently deployable teams or measured scale requirements appear
Club guard then station guard for state-changing transactions	BR-09/10	Contention; every writer must comply	NFR-04 benchmark fails or administrative batch writes become common
PostgreSQL overlap constraint as defence in depth	BR-04/05	Database-specific migration	A different DB becomes mandatory and equivalent safety is demonstrated
Durable owner + operation-key deduplication	BR-06/11	Storage and reconciliation contract	Retention/deletion policy is introduced
REST/OpenAPI	BR-03/07/08, NFR-08	Some view-specific request composition	Measured over-fetching or integration requirements justify GraphQL or push

The first implementation spike must prove migration support, concurrent creation, maintenance coordination, cancellation, and replay after response loss on actual PostgreSQL. Framework validation and an in-memory SQLite test cannot substitute for that evidence. Authentication/session choice, deployment host, exact versions and measured resource requirements remain open implementation decisions. Reviewer requested: Mansur for D4/API interaction; Nurali for D1 and administrative transaction use. Review pending.

CyberSeat - Booking and Non-functional Requirements

Version 1.0 | October 23, 2026 | Owner: Aibekuly Alibi | ECR-7

Status: design baseline prepared for review. Must = mandatory for the core system; Should = desirable for the diploma version. Acceptance statements describe future verification, not tests already passed. Sources: ECR-1 scenarios, proposal, research.

Domain semantics

A station belongs to exactly one club. A reservation identifies its owner, station, start and end instants and persisted status CONFIRMED or CANCELLED. A past visit is determined from the end instant; it is not a third stored booking status. Sending, failed and unknown are interface states. An event is informational and creates no reservation automatically.

The club-configured minimum duration is validated for new reservations (as agreed in ECR-1); its initial value requires club/user confirmation. All intervals are finite and half-open [s,e) with s < e. Two intervals overlap exactly when s1 < e2 AND s2 < e1. Times are transmitted with an offset, validated against the club's IANA zone and stored as UTC instants; the zone identifier remains part of club configuration. Ambiguous or nonexistent local times require explicit resolution, never silent guessing. Future working intervals may cross midnight and must be converted to a continuous instant interval before containment is checked.

Availability requires an active club and station, full containment within an open working interval, and no overlap with a confirmed reservation or maintenance interval. Displayed availability is advisory. At commit, the server clock must still place the start in the future.

Functional requirements and acceptance criteria

ID / priority	Requirement	Verifiable acceptance criterion	Trace
BR-01 Must	Search availability for the selected club, station type and interval	Seed one free PC, one reserved PC and one blocked console; search returns the expected station statuses for that interval, with visible club zone	SC-01, SC-02
BR-02 Must	Validate interval and station/club identity on the server	Missing fields, end <= start, duration below the configured club minimum, past start, inactive station, wrong club/station pairing or out-of-hours interval are rejected without a new reservation	SC-02, SC-03
BR-03 Must	Create one reservation for the authenticated visitor after revalidation	For a valid free interval, committed record has the authenticated owner, selected station, exact instants, unique ID and CONFIRMED; success is shown only after committed response	SC-03
BR-04 Must	Prevent overlapping confirmed bookings, including concurrent requests	Release 20 different visitors simultaneously against one free station and identical interval: exactly one success absent infrastructure faults; DB contains exactly one confirmed record. Repeat for partial overlap and containment	SC-03
BR-05 Must	Permit adjacent intervals and independent stations	Book [14:00,15:00) then [15:00,16:00) on the same station: both succeed; same interval on two stations also succeeds	SC-02, SC-03
BR-06 Must	Deduplicate retries of one logical creation operation	Two concurrent requests with the same authenticated owner and operation key and identical body return the same reservation ID; one record exists. Reusing the key for a different body returns a conflict	SC-03; common states

ID / priority	Requirement	Verifiable acceptance criterion	Trace
BR-07 Must	List and show only the visitor's own reservations	Visitor A cannot obtain B's details by changing an ID or list filter; authenticated ownership comes from the session, not a client owner field	SC-04; common states
BR-08 Must	Cancel an own confirmed booking only before its start	A valid cancellation records CANCELLED, preserves history and releases only that booking's occupancy. At/after start or for another owner it is rejected. A repeat of a completed cancellation is harmless	SC-05
BR-09 Must	Coordinate bookings with maintenance creation and changes	Race a valid reservation and overlapping maintenance for the same station: no committed state contains both. If a booking wins, maintenance is rejected; if maintenance wins, booking is rejected	SC-03, SC-10
BR-10 Must	Preserve reservations when working hours or station status change	A change that would invalidate a future confirmed booking is rejected with a conflict; it does not silently cancel or delete the booking	SC-08, SC-09
BR-11 Must	Reconcile an unknown network outcome	Drop the response after commit; refresh own reservations and retry/query using the same operation key resolves to the original ID. UI says result is being checked until reconciliation, and does not create a new operation automatically	SC-03; common states
BR-12 Must	Revalidate after stale availability or session expiry	A second client takes the displayed slot; first client receives a conflict and refreshes choices. An expired session cannot commit; after login the interval is checked again	SC-02, SC-03; common states

ID / priority	Requirement	Verifiable acceptance criterion	Trace
BR-13 Must	Distinguish loading, empty, validation, conflict and server failure	Each state has a textual message and an appropriate next action. A server error is never displayed as an empty catalogue or confirmed booking	All scenarios; common states
BR-14 Must	Show club-local time consistently	Confirmation, own history, administration and calendar show the same instant in the selected club's zone with zone label; UTC/local conversion round-trips in a controlled fixture	SC-02 through SC-06
BR-15 Must	Retain an audit trail for changes	Creation, cancellation and rejected administrative conflicts can be correlated through operation/request IDs; audit records contain actor, action, target, time and result, without credentials	SC-03, SC-05, SC-09
BR-16 Should	Offer another interval or station after a conflict	Conflict view preserves the selected club/time and allows another availability search; alternatives are not presented as already held	SC-02, SC-03

Invariants and state transitions

I1: no two confirmed reservations for one station overlap. I2: confirmed reservations and maintenance for one station do not overlap. I3: a new or changed reservation fits an open schedule at commit. I4: a logical creation operation has at most one reservation. I5: owner and assigned-club checks are server-side. I6: cancellation cannot erase history or another occupancy source.

Only CONFIRMED -> CANCELLED is a persisted user transition. Creation enters CONFIRMED atomically; rollback creates no booking. Repeated cancellation returns the existing cancelled result. Creating a new reservation after cancellation uses a new operation key; retrying the old key returns the original, now cancelled booking.

Proposed API contract (design, not implemented endpoints)

Operation	Proposed contract	Principal result/error
Availability	GET /api/v1/clubs/{id}/availability?start=...&end=...	200 advisory statuses; 422 invalid interval
Create	POST /api/v1/reservations, operation key plus station and offset-aware interval	201 first commit; 200 replay; 409 conflict; 401 session absent/expired; 422 invalid input
Own list/details	GET /api/v1/me/reservations, GET /api/v1/me/reservations/{id}	200 own data; 404 absent or foreign resource
Cancel	POST /api/v1/me/reservations/{id}/cancellation	200 cancelled/replayed result; 409 started; 404 absent/foreign
Reconcile creation	GET /api/v1/me/reservation-operations/{key}	200 committed result; 202 in progress; 404 no durable result yet, which alone is not proof that an in-flight request failed

Error bodies carry a stable machine code, safe message and correlation ID. The client cannot supply a trusted owner or role. Proposed idempotency records are unique by owner and operation key, include a normalised-body fingerprint and are committed with the booking. Retain the operation mapping for the lifetime of the booking in the MVP. A future retention limit requires an explicit protocol change. Standard HTTP POST is not assumed to be inherently idempotent.

Non-functional requirements

ID / priority	Requirement and target	Verification conditions
NFR-01 Must	Zero invariant violations in the defined concurrency suite	Real PostgreSQL, independent connections/API workers, synchronised race starts; 100 runs each for same-slot, partial overlap, booking vs maintenance, booking vs closure and cancel vs create; query DB state after each run
NFR-02 Must	No unauthorised own-record or cross-club operation	Negative tests for unauthenticated, visitor A/B, assigned/unassigned administrator and system administrator; direct HTTP requests as well as UI checks. ECR-8 defines complete administrative permissions
NFR-03 Must	Committed data survives API restart	Commit a booking, restart API, reconcile operation and read history; ID/status unchanged. Force failure before commit: neither booking nor success mapping remains
NFR-04 Must	Draft performance target: p95 availability <=500 ms, creation/cancellation <=1000 ms; unexpected HTTP 5xx <1%	Dedicated 4-vCPU/8-GB host, local Docker network, one API service and PostgreSQL; 10 clubs, 500 stations, 100,000 reservations, 30-day horizon; 50 virtual users, 80% availability/10% creates/10% cancels, 2-min warmup and 10-min measured run, three runs. Report achieved request rate and raw results; classify expected 409 conflicts separately. Targets require team agreement
NFR-05 Must	Bounded contention and errors	Draft lock timeout 2 s, maximum 3 full-transaction retries for retryable deadlock/serialization failure with bounded backoff; exhaustion returns explicit retryable failure, never success. Timeouts and retry counts must be recorded with load results
NFR-06 Must	Usable core flow at 360px and desktop width, with keyboard access	Check visible focus, meaningful labels and text statuses, field errors, no keyboard trap and no horizontal page overflow for core screens; use WCAG 2.2 AA as the design target, not a claim of certified conformance
NFR-07 Should	Draft usability goal: at least 80% unassisted task completion	Recruit at least 5 relevant visitors; each finds a free station, books and cancels. Report per-task counts, time and errors plus SUS where appropriate. Small convenience samples are formative and cannot establish population-wide benefit

ID / priority	Requirement and target	Verification conditions
NFR-08 Must	Reproducible environment and migrations	A teammate uses documented pinned dependencies, synthetic seed and migrations in a fresh environment; no undocumented manual schema edits. This check belongs to implementation, not current documentation
NFR-09 Must	Safe operational data handling	HTTPS in deployment; secrets excluded from source; credential/token values absent from error bodies and logs; server-side authorisation repeated on every operation. Authentication mechanism to be agreed in ECR-8
NFR-10 Must	Reports use defined denominators and consistent snapshots	Utilisation definition specifies booking statuses, time clipping and available station-minutes; compare with hand-calculated synthetic fixtures. Past end time alone does not prove attendance; occupancy reporting is scheduled utilisation, not measured actual usage

These numerical targets are explicit proposed evaluation conditions, not derived user evidence or measured performance. NFR-04 and NFR-07 require Nurali's evaluation-plan review and Mansur's UX review.

Boundary examples for later tests

Existing / requested data	Expected result
Confirmed [14:00,15:00) / request [14:30,15:30)	Reject overlap
Confirmed [14:00,15:00) / request [13:00,16:00)	Reject containment
Confirmed [14:00,15:00) / request [15:00,16:00)	Allow if other rules pass
Cancelled [14:00,15:00) / same interval new operation	Allow if other rules pass
Maintenance [14:00,15:00) / request same interval	Reject regardless of cancelled booking history
Open interval 22:00-02:00 next day / request 23:00-01:00	Allow after club-zone conversion if other rules pass
Booking vs concurrent closure of the same club	Serialise at shared club guard; whichever commits second sees the first
Same operation key / changed station or interval	Reject key reuse; never create second booking

Review and dependencies

Review requested: Nurali for BR-09/10, administrative write protocol and NFR-02/10; Mansur for BR-11/13/14 and interface feasibility. New-document approval is pending. ECR-8 must use the same guards for schedule/maintenance writes; ECR-13 must record agreed targets and executed test evidence later; ECR-12 must translate invariants into schema/migrations. The October deliverable is this specification with its acceptance criteria.

CyberSeat - System Context and Component Architecture

Version 1.0 | October 23, 2026 | Owner: Aibekuly Alibi | ECR-10

Status: proposed architecture prepared for review. These are design views, not deployed services. Detailed ER design is a later ECR-12 deliverable.

1. Views and editable sources

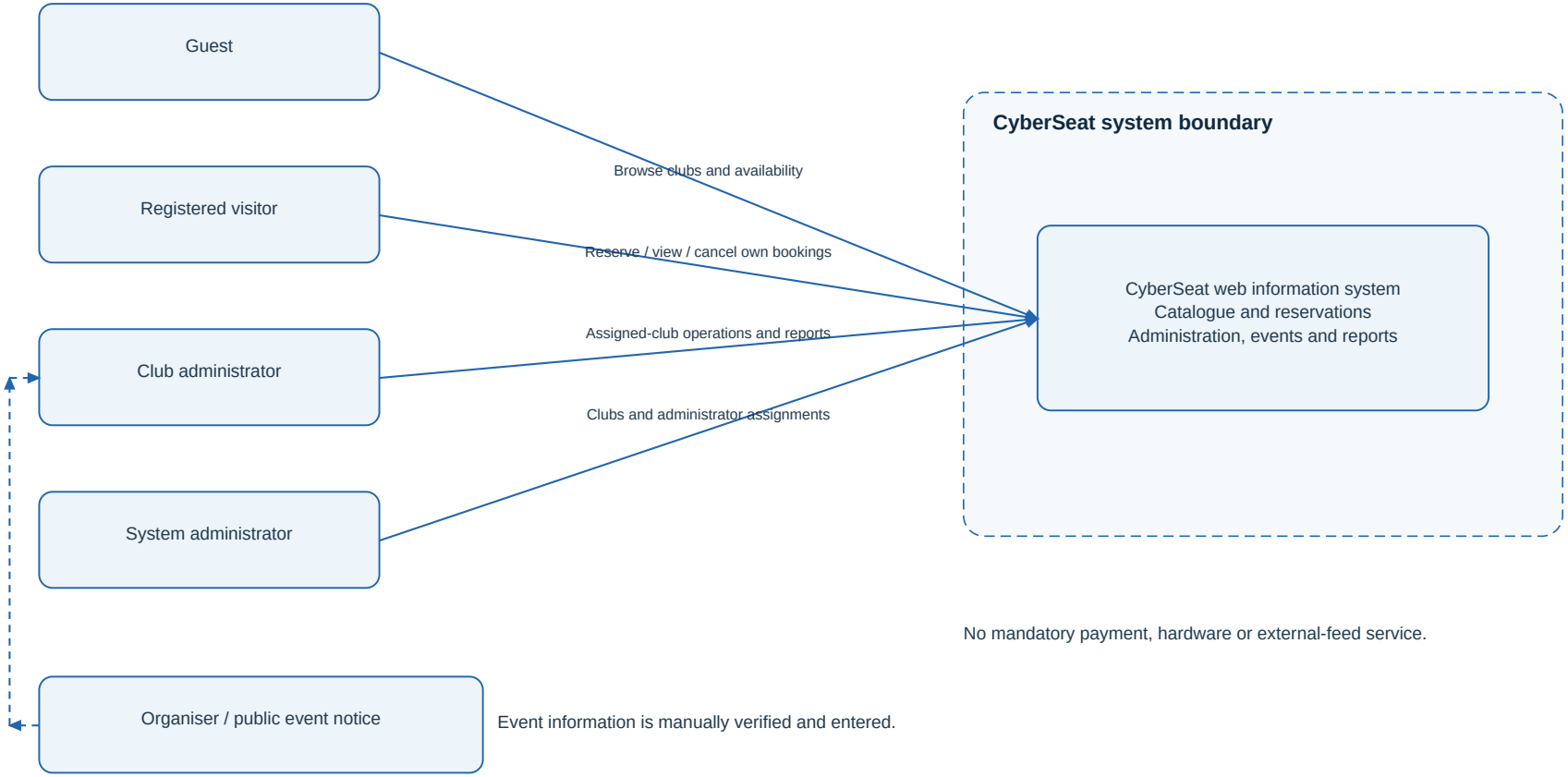
View	Editable source	Readable export	Purpose
System context	system-context.mmd	system-context.svg	People outside the CyberSeat boundary and their interactions
Component overview	components.mmd	components.svg	Web client, API boundary, logical modules and PostgreSQL

Mermaid sources and SVG exports express the same nodes and relationships. SVG labels are editable text. The component view deliberately includes client and storage context; it is a practical component overview rather than a claim of strict UML notation. Solid arrows denote user requests or runtime calls; the dashed event-source arrow denotes manual information entry. A module is a logical responsibility inside one server deployment, not an independently deployed microservice.

2. System context

CYBERSEAT / SYSTEM CONTEXT

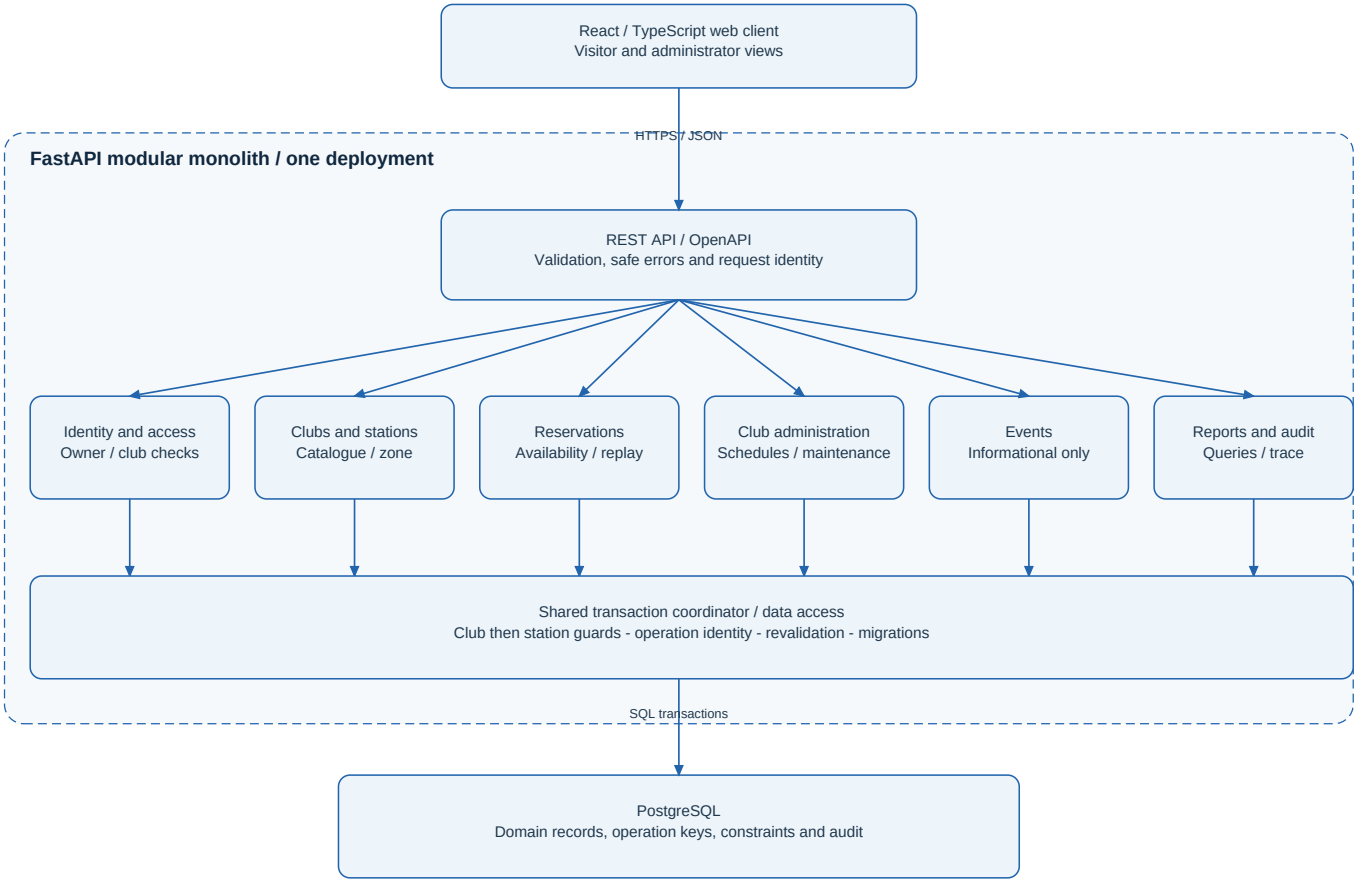
Proposed design - ECR-10 - October 23, 2026



Guests browse clubs and availability. Registered visitors also create, view and cancel only their own reservations. Club administrators manage stations, working hours, maintenance, bookings, published events and reports within assigned clubs. System administrators manage clubs and assignments. A multi-club manager is represented through assignments rather than unrestricted access by default.

An organiser or public event notice is an information source outside the system. Administrators manually verify and enter event details; there is no required external-feed API. Hosting, the browser and the database are not additional business actors. GitHub and YouTrack manage development and are not runtime dependencies of the reservation workflow. Payment providers and hardware agents are absent because those capabilities are excluded.

3. Component overview



Component	Responsibility and data	Requirement / scenario trace
React/TypeScript web client	Catalogue, selected interval, reservation forms, own history, calendar and assigned-club admin views; transient loading/error states	BR-01/11/13/14; SC-01 through SC-06; common states
REST API boundary / FastAPI	Request models, route contracts, safe errors, authenticated context, correlation/operation IDs; no trust in client owner or role	BR-02/03/06/07; SC-03; common states
Identity and access	User/session identity, visitor ownership and current club assignments; checked for every operation	BR-07/08, NFR-02; SC-07/14
Club and station catalogue	Clubs, station identity/specifications, active status and club zone	BR-01/02/14; SC-01/02/08
Reservation and availability	Availability calculation, creation, own history, cancellation and durable replay semantics	BR-03 through BR-12; SC-02 through SC-05
Club administration	Authorised station, schedule and maintenance changes; all booking-affecting writes use the same coordinator	BR-09/10; SC-07 through SC-10
Events	Draft/published/cancelled event information, source, zone and club association; no automatic reservation	SC-06/13
Reporting and audit	Defined scheduled-utilisation queries and actor/action/result records	BR-15, NFR-10; SC-12
Transaction coordinator / data access	Shared lock order, revalidation, operation uniqueness, transaction and error mapping, migrations	BR-04/06/09/10, NFR-01/03/05
PostgreSQL	Durable users, assignments, clubs, stations, schedules, maintenance, reservations, operation identities, events and audit data	All persisted invariants; future ECR-12 schema

The API invokes identity/access and the applicable domain module. Modules use the shared data-access layer; administration must not implement an independent unsafe booking writer. Reporting reads committed data and never changes reservation status. Audit writes for successful mutations share the business transaction; logging a rejected request is a separate operational event and must not turn a rejection into a committed booking.

4. Transaction protocol (proposed implementation contract)

Database baseline: PostgreSQL READ COMMITTED with explicit guard locks and a reservation overlap constraint. This is a design to validate during implementation. All steps below belong to one transaction; reads that decide a write happen after the necessary guards are acquired. A process-local mutex cannot protect independent API workers.

1. Authenticate the actor. For creation, claim a unique (owner_id, operation_key) record and check its normalised payload fingerprint. A concurrent same-key request waits for the first transaction and then reuses its result. A different payload with the same key is a conflict. The successful operation mapping and reservation commit together.

2. Acquire the club guard row FOR SHARE for a station-level operation. Working-hours, club-zone or club-status writers acquire the same guard FOR UPDATE. Club configuration writers therefore conflict with in-progress station operations; ordinary reservations at different stations may proceed concurrently.

3. Acquire the station guard row FOR UPDATE. Reservation creation, cancellation, maintenance and station-status changes use this same guard. For any batch operation, acquire clubs and stations in ascending ID order; never acquire the club guard after a station guard.

4. Recheck actor/assignment, club and station state, server time, working-hours containment, overlapping confirmed reservations and maintenance. Assignment changes must also coordinate with the relevant authorisation guard or club guard; checking an assignment before waiting for locks is not sufficient for revocation-sensitive writes.

5. Insert or update the reservation and audit record. The database additionally rejects overlapping confirmed tstzrange intervals for the same station through a GiST exclusion constraint. Cancellation removes that reservation from active occupancy but preserves its history and operation mapping.

6. Commit before returning success. Map a known interval conflict to a stable 409 domain error. On deadlock/serialization failure, retry the complete transaction within NFR-05 bounds; on exhaustion or timeout return an explicit retryable error. Never retry a known business conflict as if it were a transient fault.

For a club schedule update, the writer takes the exclusive club guard, checks all affected confirmed future bookings and rejects a change that would invalidate them. No implicit cancellation occurs. If stations in several clubs are ever changed together, take all club guards before station guards in the universal order. Club-zone changes with future bookings require explicit migration policy or rejection; silent reinterpretation of instants is prohibited.

The overlap constraint protects reservations in one table. Maintenance in another table is safe only because all applicable writers take the same guards and revalidate; the constraint is not described as enforcing a cross-table invariant. Direct manual DB writes bypassing the protocol are outside authorised application operations and must be restricted operationally. An alternative future design could unify occupancy records or use serialisable transactions, but requires its own proof and tests.

5. Runtime flows and failure boundaries

Create: browser requests availability -> API returns advisory state -> visitor confirms with an operation key -> access check and transaction protocol -> commit -> response with reservation ID. If another writer wins the station interval, the loser gets a conflict and refreshes availability. After a lost response the browser refreshes its own reservation list and reconciles using the same key rather than inventing a second operation.

Cancel: browser requests cancellation -> server verifies owner under the shared protocol -> confirmed future reservation becomes cancelled -> commit -> refreshed own history. A second cancellation returns the cancelled state. Maintenance occupancy remains independent.

Maintenance: assigned administrator requests a block -> server checks club assignment and takes guards -> conflict check against confirmed bookings -> commit only if safe. A race with reservation creation has one order, so the second writer observes the first.

Read/report: apply access filters on the server -> query committed records -> return selected fields. A station's operational availability and report denominator need defined schedule/maintenance rules. A past reservation proves planned time, not attendance. Bulk reporting should use a defined snapshot when several queries contribute to one result.

The browser is untrusted; it has no database credentials and cannot commit directly. Only the API reaches PostgreSQL. TLS, secrets, session handling and resource access are implementation concerns to verify, not proven properties of these diagrams. No external payment, event feed or notification service can prevent core booking operation because none is mandatory in this scope.

6. Consistency and review checklist

- Roles and informational-calendar semantics match ECR-1 and the current README.
- Components use FastAPI, PostgreSQL and REST proposed in ECR-6.
- BR-04/09/10 map to one common transaction coordinator.
- BR-06/11 map to durable operation identity and network-outcome reconciliation.
- NFR-02 maps to resource/club checks in the API and domain operations.
- Sources and exports are provided for both required views; ER details remain in ECR-12.

Required human review: Mansur for client/API boundaries and failure states; Nurali for administrative writes, assignments and reporting. Their approval of this new architecture is pending. The diagrams do not replace Mansur's separate ECR-11 use-case diagram.